

What I found is that the Grayscale and Binarized image processing were approx. the same speed. However, the grayscale was less accurate due to the adjustments that had to be made to `imfindcircles` to allow to be more lenient on finding the ball. While with these adjustments it found the ball consistently, its bounds on the image are smaller, and distance values slightly larger.

Binarized was seemingly more accurate, and used `regionprops` to find an approx. centroid in the region of interest. However, because of the centroid and blob method, we need to specify the region of interest quite narrowly to avoid other potential large blobs, such as the card seen in the test images.

YCbCr was the slowest, and least reliable method of identifying the ball. I attempted multiple methods of processing the image splitting the YCbCr channels and had various results, for my app I ended up using the luminosity channel Y with a region of interest mask and `imfindcircles`. This produced the most consistent results but leads to an issue of any high luminosity area being perceived as a circle. I know I can tune this further and make it the most accurate method though, however additional tuning and image processing will make it take even longer, which is something that I do not think is worth it in this case. For instance, one method I used broke the image down into the three channels, and processed the Cr and Cb channels twice with a spatial mask. This resulted in the card within the image being a dark blue color, and the ball being a bright red, which was very easy for `imfindcircles` and the centroid method to detect, but because I was doing additional levels of processing, it took significantly longer and I deemed it a step too far.

In the future I want to adjust the GUI a bit more and allow for further user input on the adjustments, such as letting the user select a bounding box region, `imfindcircles`, or both. And have further customization with each of those, such as selecting specific pixel regions for the region of interest, or adjusting the `imfindcircles` sensitivity from the gui and not having it hard coded. These are methods and adjustments I will keep in mind for the future for better analysis and system testing, but for now I will omit them.